UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Contrôler qui accède à quoi

CL-SECU-API

API, ce que chacun peut y faire, et ce qu'elle laisse sortir.

Utopios® Tous droits réservés

Objectifs de la journée

À la fin de la journée, vous saurez :

- **Expliquer** pourquoi les API sont visées, avec trois incidents publics, et **situer** les 10 risques OWASP API 2023.
- **Exiger** un jeton signé et de courte durée sur chaque route, avec des mots de passe hachés et un secret ni dans le code ni dans l'image.
- **Limiter** chaque utilisateur à ses objets et à ses fonctions, et répondre 401, 403 ou 404 à bon escient.
- **Choisir** les champs qui entrent dans l'API et ceux qui en sortent.
- **Prouver** chaque protection par des tests automatisés, en particulier des tests de **refus**.

Utopios® Tous droits réservés

1. Pourquoi les API

Des portes d'entrée directes vers les données

Utopios® Tous droits réservés

Une API REST en une diapo

Une **API** (interface de programmation) permet à un programme d'en appeler un autre. Une API **REST** passe par HTTP : une **URL** désigne une ressource, une **méthode** dit l'action, la réponse arrive en **JSON** avec un **code**.

Requête à StockLine	Ce qu'elle fait
`GET /produits`	lister les produits
`GET /produits/4`	lire le produit n° 4
`POST /mouvements` + corps JSON	enregistrer une entrée de stock
`DELETE /produits/6`	supprimer le produit n° 6

Code	Sens
200 · 201 · 204	OK · créé · supprimé
401	non authentifié
403	interdit
404	introuvable
422	données refusées

FastAPI publie la description de toutes ses routes au format **OpenAPI**, affichée par la page **Swagger** (`/docs`) : pratique pour les développeurs… et pour quiconque cherche un point faible.

Utopios® Tous droits réservés

Pourquoi les API attirent les attaques

Caractéristique	Pourquoi c'est un risque
Elles exposent la **logique métier** et les **données** brutes	aucun écran ne filtre : le client reçoit tout ce que l'API renvoie
Les **identifiants** sont dans l'URL (`/produits/4`)	changer un numéro suffit à demander l'objet d'un autre
Tout est **automatisable**	un script enchaîne des milliers de requêtes, sans se lasser
Elles sont **documentées** (OpenAPI)	la carte des routes et des champs est souvent publique
Elles se **multiplient** : mobile, partenaires, microservices	des versions oubliées restent en ligne, sans protection
Leur trafic est du **HTTPS valide**	un pare-feu réseau ne distingue pas l'usage normal de l'abus

Conséquence : le contrôle doit se faire **dans l'API**, à **chaque** requête. Le réseau ne suffit pas.

Utopios® Tous droits réservés

Trois incidents publics

	Optus (Australie, 2022)	T-Mobile (États-Unis, annoncé en janvier 2023)	Peloton (2021)
Les faits	Selon les rapports publics, une API joignable depuis Internet répondait **sans authentification**.	Selon l'opérateur, un tiers a obtenu des données **via une seule API, sans autorisation**, de fin novembre 2022 à début janvier 2023.	Selon le chercheur qui l'a signalée, l'API renvoyait des données de profils, même réglés « privés », à des requêtes **non authentifiées**.
L'ampleur	environ **10 millions** de clients (jusqu'à 9,8 millions selon Optus) : noms, dates de naissance, coordonnées, et numéros de pièce d'identité pour une partie	environ **37 millions** de comptes : noms, adresses, téléphones, dates de naissance ; ni mots de passe ni cartes bancaires, selon l'opérateur	âge, sexe, ville, poids, statistiques d'entraînement
La leçon	exiger une identité sur **chaque** route (API2)	une API qui « fonctionne » peut être détournée : limiter ce qu'elle renvoie, **surveiller** son usage (jour 2)	un premier correctif a seulement exigé une connexion : tout membre connecté lisait encore les profils des autres. S'authentifier ne suffit pas (API1)

Point commun : les données sont sorties par des **requêtes d'API**, pas par une intrusion dans les serveurs. La protection doit donc vivre **dans l'API**.

Utopios® Tous droits réservés

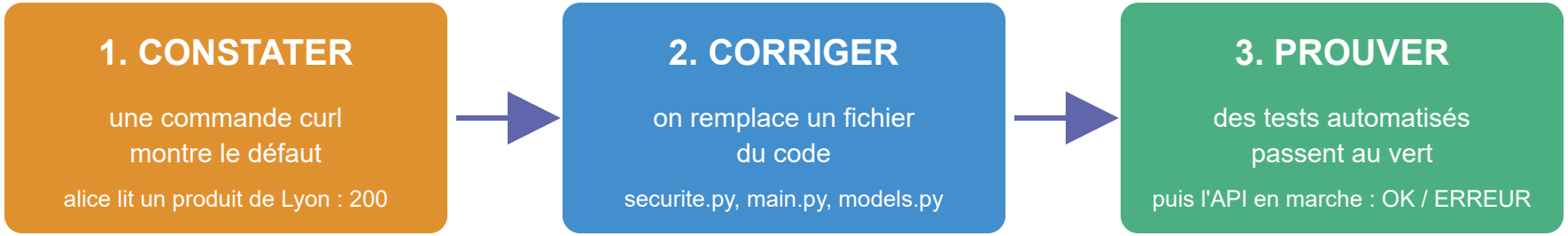
Le Top 10 OWASP des API (2023)

Risque	En une phrase
API1 — Autorisation défaillante au niveau des objets (*BOLA*)	L'API ne vérifie pas que l'objet demandé est dans le périmètre de l'appelant.
API2 — Authentification défaillante	Jetons, mots de passe ou sessions mal protégés : on se fait passer pour un autre.
API3 — Autorisation défaillante au niveau des propriétés des objets (*BOPLA*)	L'API renvoie ou accepte des champs que le client ne doit ni voir ni fixer.
API4 — Consommation de ressources illimitée	Ni limite de taille, ni de débit, ni de coût : l'API s'épuise ou coûte cher.
API5 — Autorisation défaillante au niveau des fonctions (*BFLA*)	Une fonction réservée (supprimer, administrer) est ouverte à un rôle qui ne devrait pas l'avoir.
API6 — Accès illimité aux flux métier sensibles	Un parcours métier (achat, réservation, inscription) est automatisé à grande échelle.
API7 — Falsification de requêtes côté serveur (*SSRF*)	L'API va chercher une URL fournie par le client, y compris vers le réseau interne.
API8 — Mauvaise configuration de sécurité	Réglages par défaut, erreurs trop bavardes, CORS trop large, HTTPS absent.
API9 — Gestion défaillante de l'inventaire	Des API ou des versions oubliées restent en ligne, sans surveillance.
API10 — Consommation non sécurisée d'API	Les réponses d'une API tierce sont utilisées sans vérification.

Intitulés officiels en anglais sur owasp.org ; le module utilise ces traductions. *BOLA*, *BOPLA*, *BFLA* : les sigles anglais d'API1, API3 et API5.

Utopios® Tous droits réservés

Le cycle de chaque demos



	Départ	Demo 1	Demo 2	Demo 3
Tests au vert (`./lancer-tests.sh`)	4	17	24	28

- Aux exercices 2 et 3, les tests sont écrits **avant** la correction : ils échouent d'abord, preuve qu'ils détectent le défaut.
- Les 4 tests fonctionnels du départ restent verts jusqu'au bout : sécuriser une API ne doit pas la casser.

Utopios® Tous droits réservés

2. Authentification (API2)

Savoir qui appelle, à chaque requête

Utopios® Tous droits réservés

Authentification ou autorisation ?

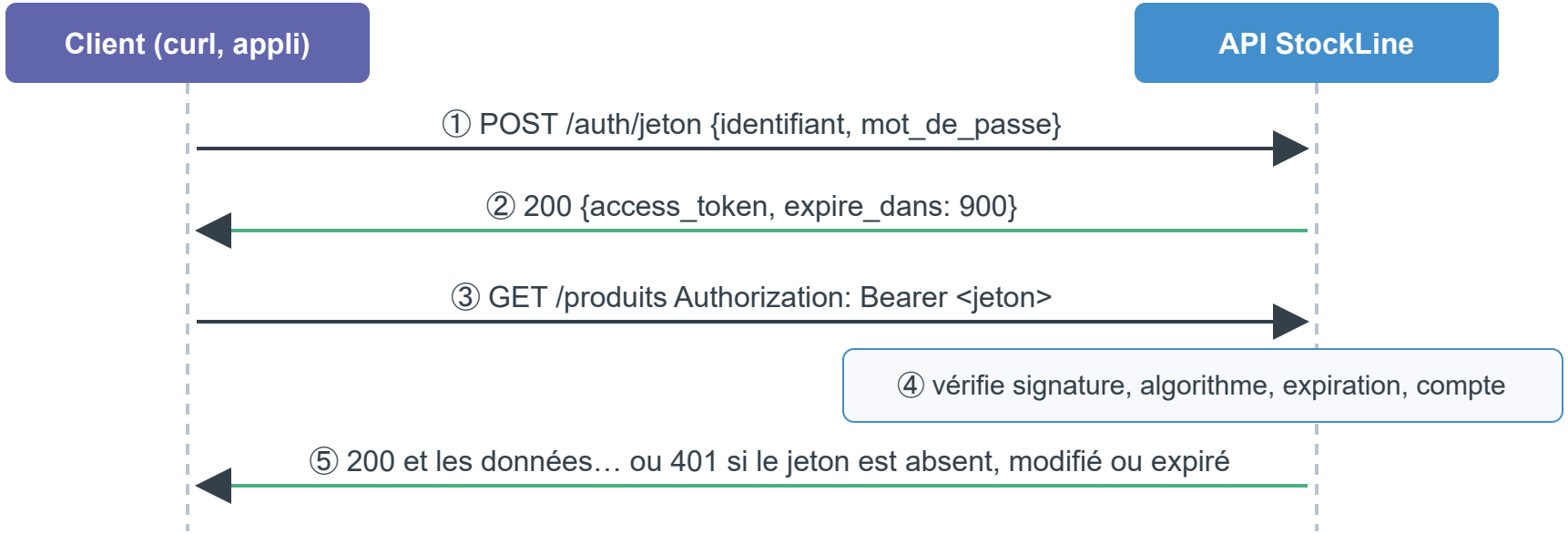
	Authentification	Autorisation
La question	Qui êtes-vous ?	Avez-vous le droit ?
Ce qui le prouve	un mot de passe, puis un jeton	un rôle, un entrepôt, la propriété de l'objet
Le refus	**401** : « identifiez-vous »	**403** : « pas pour vous » (ou 404, on y revient)
Risque OWASP	API2	API1, API3, API5
Exercice	1	2 et 3

Piège de vocabulaire : le nom officiel du code 401 est *Unauthorized*, mais il signifie en réalité « **non authentifié** ».

On authentifie d'abord : impossible de décider ce qu'une personne peut faire sans savoir qui elle est.

Utopios® Tous droits réservés

Le jeton : un badge temporaire



Le mot de passe ne voyage **qu'une fois**. Ensuite, le jeton est présenté à **chaque** appel. Toujours en **HTTPS** : pendant ses 15 minutes de validité, un jeton intercepté vaut le mot de passe.

Utopios® Tous droits réservés

Anatomie d'un jeton JWT

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhbGljZSIsImlhdCI6MTc5MzAwMDAwMCwiZXhwIjoxNzkzMDAwOTAwfQ.‹signature›
```

Partie	Dans StockLine	À quoi elle sert
En-tête	`{"alg": "HS256", "typ": "JWT"}`	annonce l'algorithme de signature
Contenu	`{"sub": "alice", "iat": 1793000000, "exp": 1793000900}`	qui (`sub`), émis à (`iat`), expire à (`exp`) : 900 s, soit 15 min
Signature	calculée avec le **secret** du serveur	prouve que l'en-tête et le contenu n'ont pas été modifiés

- **Encodé n'est pas chiffré** : en-tête et contenu sont lisibles par n'importe qui. **Jamais de donnée sensible** dans un jeton.
- Un seul caractère modifié casse la signature : **401**.
- Le rôle et l'entrepôt ne sont **pas** dans le jeton : le serveur les relit dans sa liste de comptes, à chaque appel.

Utopios® Tous droits réservés

Trois règles pour un jeton fiable

- **Expiration courte** : `DUREE_JETON` vaut 15 minutes. Un jeton volé ne sert pas longtemps.
- **Algorithme imposé** à la vérification : `algorithms=[ALGORITHME]`. Un jeton qui annonce un autre algorithme, ou aucun, est refusé.
- **Secret long, ni dans le code ni dans l'image** : 32 caractères minimum, fourni par un **secret Docker** (diapo suivante). Sans lui, l'API refuse de démarrer.

Chaque route déclare `Depends(utilisateur_courant)` : FastAPI exécute ce contrôle **avant** la route. Seules `/sante` et `/auth/jeton` restent publiques.

```python
def utilisateur_courant(credentials=Depends(_schema_bearer)):
    """Dépendance FastAPI : exige un jeton valide, sinon 401."""
    if credentials is None:
        raise refus
    try:
        # algorithms est imposé : un jeton signé autrement
        # est refusé.
        charge = jwt.decode(
            credentials.credentials,
            SECRET,
            algorithms=[ALGORITHME],
            options={"require": ["exp", "sub"]},
        )
    except jwt.PyJWTError:   # signature fausse, jeton expiré…
        raise refus from None
    compte = UTILISATEURS.get(charge["sub"])
    if compte is None:
        raise refus
    # Le rôle et l'entrepôt viennent du serveur, pas du jeton.
    return Utilisateur(identifiant=charge["sub"],
                       role=compte["role"],
                       entrepot=compte["entrepot"])
```

Utopios® Tous droits réservés

Le secret de signature : un secret Docker

```
# [VM srv-linux] : fichier lisible par vous seul
mkdir -p secrets
(umask 077 && openssl rand -hex 32 > secrets/jwt_secret.txt)
```

```
# compose.yaml (extrait)
services:
  api:
    environment:
      STOCKLINE_JWT_SECRET_FILE: /run/secrets/jwt_secret
    secrets:
      - jwt_secret

secrets:
  jwt_secret:
    file: ./secrets/jwt_secret.txt
```

- **Tiré au hasard** : `openssl rand -hex 32` donne 64 caractères hexadécimaux. Avec `umask 077`, le fichier n'est lisible que par vous (`-rw-------`).
- **Monté en lecture seule** dans le conteneur, sous `/run/secrets/jwt_secret`. L'application trouve ce chemin dans `STOCKLINE_JWT_SECRET_FILE`.
- **Ni dans le code, ni dans l'image** : `.dockerignore` exclut le dossier `secrets/`. L'image peut être partagée sans livrer le secret.
- Contrairement à une variable d'environnement, sa valeur n'apparaît pas dans `docker inspect`.

Fichier absent, ou secret de moins de 32 caractères : l'API **refuse de démarrer**. Mieux vaut une panne visible qu'un secret faible.

Utopios® Tous droits réservés

Les mots de passe : hachés, et sans indice

Stockage		Pourquoi
En clair	❌	une fuite de la base livre tous les mots de passe
Chiffré, avec la clé dans l'application	❌	qui obtient la clé déchiffre tout
Haché en SHA-256 simple	❌	trop **rapide** : une carte graphique teste des milliards de mots de passe par seconde
PBKDF2, **bcrypt** ou **Argon2**, avec un **sel**	✅	**lent** exprès ; le sel, propre à chaque compte, donne deux empreintes différentes à deux mots de passe identiques

StockLine : PBKDF2-SHA256, **600 000 itérations** (valeur recommandée par l'OWASP), un sel par compte, comparaison en temps constant (`hmac.compare_digest`).

Ne rien révéler à la connexion :

- même message, « Identifiant ou mot de passe incorrect. », que le compte existe ou non ;
- même calcul, donc même temps de réponse, si le compte n'existe pas.

Limiter le nombre d'essais : vue plus tard.

Utopios® Tous droits réservés

En production : un fournisseur d'identité

Le labo gère trois comptes dans le code, pour que vous voyiez chaque mécanisme. En production, on ne réinvente pas l'authentification.

	Labo StockLine	Production
Comptes	un dictionnaire dans `securite.py`	un **fournisseur d'identité** : Keycloak, Microsoft Entra ID, Okta…
Qui délivre le jeton	l'API elle-même (`POST /auth/jeton`)	le fournisseur d'identité, selon **OAuth 2.0** et **OpenID Connect**
Signature	HS256 : un secret partagé	souvent RS256 : clé privée chez le fournisseur, clé publique pour vérifier
MFA, verrouillage, mot de passe oublié	absents	pris en charge par le fournisseur d'identité
Ce que vérifie l'API	signature, algorithme, expiration	les mêmes, plus l'émetteur (`iss`) et le destinataire (`aud`)

- **OAuth 2.0** : le protocole standard qui délivre des **jetons d'accès** à une application.
- **OpenID Connect** : une couche au-dessus d'OAuth 2.0 qui dit **qui** est l'utilisateur (jeton d'identité).

Utopios® Tous droits réservés

3. Autorisation (API1 et API5)

Chacun ses objets, chacun ses fonctions

Utopios® Tous droits réservés

Objet ou fonction : deux contrôles

Un utilisateur **authentifié** n'est pas pour autant **autorisé**. Deux vérifications, à chaque requête :

	API1 — au niveau des objets (*BOLA*)	API5 — au niveau des fonctions (*BFLA*)
La question	Cet objet-là est-il dans votre périmètre ?	Cette action est-elle permise à votre rôle ?
Dans StockLine	alice (Paris) lit le produit 4, rangé à Lyon	alice supprime un produit, action réservée au rôle `admin`
Avant correction	200 : le produit s'affiche	204 : le produit est supprimé
Après	**404**, comme un produit inexistant	**403** : la fonction existe, pas pour ce rôle
Où vérifier	sur **chaque** objet lu, modifié ou listé	sur **chaque** route sensible

API1 est en tête du Top 10 depuis sa première édition (2019) : le contrôle est facile à oublier sur une seule route.

Utopios® Tous droits réservés

Un identifiant n'est pas une protection

Des numéros qui se suivent (1, 2, 3…) : il suffit de compter pour tomber sur les objets des autres.

Des identifiants aléatoires (UUID) : plus difficiles à deviner, mais ils **circulent** : liens partagés, journaux, historique du navigateur, autres réponses de l'API…

Un UUID réduit le risque ; il ne **remplace** jamais un contrôle. Un identifiant n'est pas un secret.

La règle

- Vérifier le droit sur l'objet **à chaque requête**, même si l'identifiant « ne devrait pas » être connu.
- Au **même endroit** pour toutes les routes : une seule fonction, appelée partout. Un contrôle oublié sur une seule route suffit à ouvrir l'accès.
- Le périmètre vient du **serveur** (le compte derrière le jeton), jamais d'un paramètre envoyé par le client.

Utopios® Tous droits réservés

Le contrôle, écrit une seule fois

```python
def _produit_autorise(session, produit_id, utilisateur):
    """Le produit existe ET l'utilisateur y a droit, sinon 404."""
    produit = session.get(ProduitDB, produit_id)
    autorise = produit is not None and (
        utilisateur.role == "admin"
        or produit.entrepot == utilisateur.entrepot
    )
    if not autorise:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Produit {produit_id} introuvable.",
        )
    return produit
```

Les listes sont filtrées **dans la requête SQL** :

```python
requete = select(ProduitDB).order_by(ProduitDB.id)
if utilisateur.role != "admin":
    requete = requete.where(ProduitDB.entrepot == utilisateur.entrepot)
```

- Appelée par **toutes** les routes qui reçoivent un identifiant de produit : lecture, stock, mouvement, suppression.
- **404 plutôt que 403** : un 403 confirmerait que le produit 4 existe. Avec 404, l'API ne révèle rien de ce qui est hors du périmètre.
- **Filtrer en SQL** : la base ne renvoie que ce qui est autorisé. Trier après coup en Python, c'est lire les données des autres… et risquer d'en oublier un cas.

Utopios® Tous droits réservés

Les rôles, et des tests de refus

```python
def exiger_role(*roles: str):
    """Dépendance FastAPI : réserve une route à certains rôles, sinon 403."""
    def verifier(utilisateur: Utilisateur = Depends(utilisateur_courant)) -> Utilisateur:
        if utilisateur.role not in roles:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
                                detail=f"Action réservée au rôle : {', '.join(roles)}.")
        return utilisateur
    return verifier

# Dans supprimer_produit() :   utilisateur: Utilisateur = Depends(exiger_role("admin"))
```

```python
def test_suppression_reservee_a_l_admin(client):
    assert client.delete("/produits/1", headers=entetes(client, "alice")).status_code == 403
    assert client.get("/produits/1", headers=entetes(client, "alice")).status_code == 200
    assert client.delete("/produits/1", headers=entetes(client, "admin")).status_code == 204
```

- `exiger_role` s'appuie sur `utilisateur_courant` : l'authentification est faite d'abord, le rôle est vérifié ensuite.
- Un **test négatif** prouve que l'interdit est refusé (403, 404). C'est lui qui compte : un test positif ne voit jamais une porte restée ouverte.

Utopios® Tous droits réservés

4. Propriétés des objets (API3)

Choisir les champs qui entrent et qui sortent

Utopios® Tous droits réservés

Deux défauts, un même risque

L'utilisateur a le droit sur l'objet… mais pas sur **tous ses champs**.

	Exposition excessive de données	Affectation de masse (*mass assignment*)
Sens	ce qui **sort** de l'API	ce qui **entre** dans l'API
Dans StockLine	`prix_achat`, donnée commerciale interne, renvoyé à chaque client	alice envoie `"entrepot": "lyon"` et crée un produit hors de son périmètre
Cause	le schéma de sortie hérite du schéma d'entrée : tout ce qui entre ressort	l'API recopie le corps de la requête dans l'objet, champ par champ
Protection	schéma de **sortie** en liste blanche (`response_model`)	schéma d'**entrée** en liste blanche (`extra="forbid"`), champ fixé par le serveur
Preuve	`prix_achat` absent de toutes les réponses	champ en trop : **422**

Ailleurs, les exemples classiques d'affectation de masse sont des champs comme `role`, `est_admin` ou `solde` acceptés sans contrôle.

Utopios® Tous droits réservés

Des schémas en liste blanche

Ce que le client a le droit d'envoyer

```python
class ProduitCreate(BaseModel):
    """Pas d'entrepôt : le serveur le fixe."""

    model_config = ConfigDict(extra="forbid")

    reference: str = Field(min_length=1, max_length=50)
    nom: str = Field(min_length=1, max_length=200)
    prix_unitaire: float = Field(gt=0)
    prix_achat: float = Field(gt=0)
    seuil_alerte: int = Field(default=5, ge=0)
```

`extra="forbid"` : tout champ non listé est refusé (422).

Ce que l'API a le droit de renvoyer

```python
class Produit(BaseModel):
    """Pas de prix d'achat, donnée interne."""

    model_config = ConfigDict(from_attributes=True)

    id: int
    reference: str
    nom: str
    prix_unitaire: float
    seuil_alerte: int
    entrepot: str
    cree_le: datetime
```

Avant : `class Produit(ProduitCreate)`. La sortie héritait de l'entrée, `prix_achat` compris.

Utopios® Tous droits réservés

Le serveur décide, le front ne protège rien

```
def creer_produit(produit: models.ProduitCreate, ...,
                  utilisateur: Utilisateur = Depends(exiger_role("gestionnaire"))):
    ...
    nouveau = ProduitDB(**produit.model_dump(), entrepot=utilisateur.entrepot)
```

Si alice envoie quand même `"entrepot": "lyon"`, la réponse 422 désigne le champ en trop :

```
{"champ":"entrepot","message":"Extra inputs are not permitted"}
```

- Une valeur qui décide d'un droit (entrepôt, propriétaire, rôle) vient **toujours du serveur**, ici du compte derrière le jeton.
- « Le front n'affiche pas ce champ » ne protège rien : la réponse JSON est lisible par quiconque appelle l'API. Masquer à l'écran n'est pas ne pas envoyer.
- Avec `response_model`, une colonne ajoutée demain en base ne sortira pas tant que personne ne l'ajoute **volontairement** au schéma.
- Refuser (422) plutôt qu'ignorer : le client voit son erreur tout de suite. En contrepartie, un client mal écrit est bloqué : à documenter.

Utopios® Tous droits réservés

La chaîne de contrôle d'une requête



Vérifié sur StockLine : les contrôles s'enchaînent **dans cet ordre**, et le premier qui échoue arrête la requête.

Une protection n'existe que si elle est **prouvée** : chaque refus de cette chaîne a son test automatisé.

Utopios® Tous droits réservés

Récap : risque, mesure, preuve

Risque	Mesure dans StockLine	Preuve
API2 — Authentification	jeton JWT de 15 min, algorithme imposé, secret Docker de 32 caractères minimum, ni dans le code ni dans l'image ; mots de passe en PBKDF2 (600 000 itérations)	sans jeton, jeton expiré ou signé avec un autre secret : **401**
API1 — Objets	`_produit_autorise()` sur chaque identifiant ; listes filtrées dans la requête SQL	alice demande un produit de Lyon : **404**, comme un produit inexistant
API5 — Fonctions	`exiger_role("admin")` sur la suppression, `exiger_role("gestionnaire")` sur la création	alice supprime : **403** ; l'admin : **204**
API3 — Propriétés	`response_model` sans `prix_achat` ; `extra="forbid"` ; entrepôt fixé par le serveur	`prix_achat` absent ; champ en trop : **422**

28 tests au vert, dont les 4 tests fonctionnels du départ : l'API est protégée, et elle rend toujours le même service.

Utopios® Tous droits réservés

Questions ?

Protéger et surveiller en production

Chaque requête a été **identifiée** et **contrôlée**.

Question à garder en tête : que se passe-t-il si un client **authentifié et autorisé** demande tous les produits d'un coup, ou essaie mille mots de passe à la suite ?

Utopios® Tous droits réservés